Atlas

Parametric flood protection for Houston

Fast cash in the first 72 hours, triggered by public gauge data, settled on Solana, priced on Tiger Data

---

20-Hour Build Plan — Version 3.1

CodeRED: Orion | Tracks: Overall, Pitch, Environment, Solana, Tiger Data, Vultr, GoDaddy

Team: Jesse (chain, product, hosting) and Ezekiel "Zeke" (data, risk); TypeScript and Python only (no Rust)

As of October 10, 2026, Hour 4 (5:00 pm CDT)

The one-sentence pitch

Maria rents in Meyerland. When her bayou crosses flood stage, $500 lands on her phone before the water goes down. That pays for a hotel, food and a missed shift in the first 72 hours, when nothing else pays.

The bayou is named on stage only after the data confirms it triggers in the replay window. Brays Bayou is preferred because Meyerland sits on it.

Assumptions: a team of two, a 20-hour build inside a 24-hour clock that started at 1:00 pm CDT on October 10, devnet only, no custom Solana program. **[VERIFY]** items must be checked by Gate G1. Tags such as **[V3]** point to the v3.1 fix list; tags such as [F7] are v3 fixes, kept.

Contents

Changes from v3 **3**

0 Status at Hour 4 **4**

1 Product summary and scope **4**

1.1 The pitch, led by a person . . . 5
1.2 The problem . . . 5
1.3 What Atlas does . . . 5
1.4 How each sponsor technology is used . . . 5
1.5 Tracks entered . . . 5
1.6 In scope . . . 6
1.7 Out of scope . . . 6

2 System architecture **6**

2.1 Data flow for one storm . . . 6

3 Pre-flight checklist (remaining items) **7**

4 Team roles and the API contract **7**

4.1 Roles . . . 7
4.2 API contract (`docs/api.md` is the source of truth) **[V1]** . . . 8

5 Timeline, re-based from Hour 4.5 [V17] **9**

6 Data layer: Tiger Data **10**

6.1 Sources and checks . . . 10
6.2 Schema . . . 10
6.3 Continuous aggregates and columnstore . . . 11
6.4 Event logic . . . 12
6.5 Run clock and replay . . . 12
6.6 Reset . . . 12

7 Pricing and capital model **12**

7.1 Event frequency . . . 12
7.2 Premium . . . 12
7.3 Correlated capital and k . . . 12

7.4 Waterfall . . . 13
7.5 Storm payouts: interim and true-up . . . 13
7.6 Two pool panels . . . 13
7.7 Ledger entries **[V4] [V10]** . . . 13
7.8 Reconciliation per run **[V3]** . . . 13
7.9 Underwriting rules . . . 13

8 Solana layer (Jesse) 14
8.1 aUSD . . . 14
8.2 Policy tokens . . . 14
8.3 Atomic purchase . . . 14
8.4 Pool: Squads multisig . . . 15
8.5 Payout bot . . . 15
8.6 Idempotency . . . 15
8.7 Evidence **[V2]** . . . 15
8.8 Bond token . . . 15
8.9 Reset (chain side) . . . 15
8.10 Upgrades (after G2, only one) . . . 16

9 Backend, frontend and hosting 16
9.1 Services . . . 16
9.2 Frontend . . . 16
9.3 Hosting, security and abuse limits . . . 16

10 Demo script 17
10.1 90-second version . . . 17
10.2 4-minute version . . . 17

11 Judge Q&A preparation 18

12 Risk register, fallbacks and cut order 18

13 Limitations of the product 19

14 Submission checklist 19

15 Unverified assumptions (G1, overdue) 20

Sources 20

Changes from v3

v3.1 keeps the v3 design and fixes the gaps found in its audit and while building the chain side. The chain service, payout bot and dashboard are built to this plan and tested end to end (section 0). The remaining work is the engine, devnet, hosting and the phone test.

#	v3 gap	v3.1 fix	Sec.
Would break a gate or the demo			
V1	Code and `docs/api.md` used v2 names while v3 renamed tables, fields, statuses, routes and the Memo prefix	v3 names adopted everywhere; `docs/api.md` is the single source of truth	4.2
V2	Evidence time format undefined across languages (JavaScript adds milliseconds, Python writes `+00:00`); no test vector	`event_time` is the event's `opened_at`, formatted with exact code in both languages; test vector in `docs/evidence.md`	8.7
V3	Reconciliation fails from run 2: reset only tops up, so premiums accumulate in the vault while the ledger restarts	Reconcile per run against opening balances read from `GET /chain/balances` at Start	7.8
V4	Purchases happen before Start, but every ledger row needs a run	Movements outside a running run are booked to run `session`	7.7
V5	Flow B co-signed whatever the client posted, so a buyer could zero the premium	Server stores the message hash it built; submit co-signs only on an exact match	8.3
V6	aUSD name and symbol impossible on a classic SPL mint without Metaplex	Cut; the phone view is the money moment	8.1
V7	Sales could stay frozen between runs	No run running means sales open	7.9
High			
V8	True-up never runs on-chain in the demo (pool is large, so $f_{\text{int}} = 1$)	One rehearsal with `FORCE_F_INT=0.6` on devnet; already passed on a local validator	5, 12
V9	Person A carried data, pricing, ledger, reconciliation and all hosting	Hosting, Caddy, domain and deploys move to Jesse, whose code is done	4.1
V10	Ledger entries and accounts undefined ("SQL functions agreed in Hour 1")	Entry table fixed; chain service writes, engine reads; `investors` account added	7.7
V11	No endpoint for the current run and clock	`GET /replay/status` added	4.2
V12	`payout_intents` could not tell "expired" from "in flight" after a restart	Adds `last_valid_block_height`, `attempts`, `event_id`, `owner`, `gauge_id`	6.2, 8.6
V13	Who marks a storm `trued_up` was unstated	The bot, once every true-up confirms	7.5

#	v3 gap	v3.1 fix	Sec.
V14	Policy activity judged at "the run clock", which moves while the bot works	Judged at the event's `opened_at`	6.5
Consistency			
V15	Section 9 still described Next.js	Web is Vite + React; Next.js notes removed	9
V16	Names dropped from the title; banner needs the gauge name	Names restored; `/policies` returns `gauge_name` and `last_payout`	9.2
V17	Late start: plan adopted at Hour 4	Timeline re-based from Hour 4.5; G2 moves to Hour 6.5; upgrades cut to one	5

0 Status at Hour 4

Built and tested (Jesse). Chain service, payout bot and dashboard, on a local validator with the devnet Squads program cloned (the devnet faucet refused the build machine).

Area	Result
Policy token	NonTransferable + MetadataPointer + TokenMetadata + PermanentDelegate on one mint; create 985 bytes, purchase 670 of 1,232; name and status `ACTV` → `PAID` → `ACTV`
Purchase	Flow A and Flow B both work; a tampered Flow B transaction (premium set to 0) is refused
Abuse limits	Second reservation per wallet refused; non-allowlisted wallet refused; rate limit at 5 per minute; reset refused without the token or via a proxy
Storm payouts	$f_{\text{int}} = 0.6$: interim 300 to three policies; $f_{\text{fin}} = 1.0$: true-up 200 each; storm `trued_up`
Crash recovery	A lost payout intent was expired and rebuilt; paid exactly once
Ledger	Every entry sums to zero; float 2,000 − 1,500 = 500 matched the chain
Reset and run 2	Float refilled by minting; policies back to `ACTV`; the same policies paid again
Evidence	Python-written preimage accepted by the bot; identical test-vector hash in both languages
Phone view	"+$500 • PAID • Brays Bayou" appeared live on a confirmed payout

Not done. Engine (schema, ingestion, events and storms, pricing, FastAPI); devnet setup; hosting and domain; Phantom phone test; G2.

1 Product summary and scope

Atlas pays a fixed sum to every covered household near a Houston bayou gauge within minutes of that gauge crossing flood stage. No claim form, no adjuster.

1.1 The pitch, led by a person

Maria rents in Meyerland. When her bayou crosses flood stage, $500 lands on her phone before the water goes down. That pays for a hotel, food and a missed shift in the first 72 hours, when nothing else pays. Keep the technology out of the first 30 seconds. Name the bayou only after it is confirmed as a demo gauge [F22].

1.2 The problem

US flood insurance runs mostly through the federal program, which has a waiting period before cover starts and claims that take weeks or months. Many Houston renters and homeowners have no flood cover at all. In the first 72 hours people need cash for a hotel, a car, food and lost wages.

1.3 What Atlas does

Atlas sells a small policy tied to the nearest stream gauge. If that gauge crosses a set flood level during the term, the policyholder gets a fixed payout (for example $500). In a storm big enough to strain the pool, part is paid immediately and the rest when the storm closes [F7]. Atlas does not pay for property damage and does not replace flood insurance.

1.4 How each sponsor technology is used

Sponsor	Use
Tiger Data	Gauge history and live readings as hypertables; continuous aggregates; columnstore compression; event and storm state; pricing from correlated historical-year scenarios; the money ledger; replay engine; simulated 2,000-policy book
Solana	aUSD test stablecoin; non-transferable Token-2022 policies with on-chain metadata; atomic purchase; Squads multisig vault; payout bot with intent records; Memo evidence with a canonical hash; catastrophe bond token. Optional: Solana Pay QR or Blink
Vultr	Hosts the engine, chain service, bot and dashboard over HTTPS
GoDaddy Registry	Domain for the hosted site

1.5 Tracks entered

Best Overall, Best Pitch/Demo, Best Environment Hack, Best Use of Solana, Best Use of Tiger Data, Best Use of Vultr, Best Domain Name from GoDaddy Registry.

Side-track rule [F24]

Read the Vultr and GoDaddy Registry rules now. Side-track work is capped at 1 hour total beyond hosting the site and pointing the domain. Confirm on Devpost that one project can enter all seven tracks.

1.6 In scope

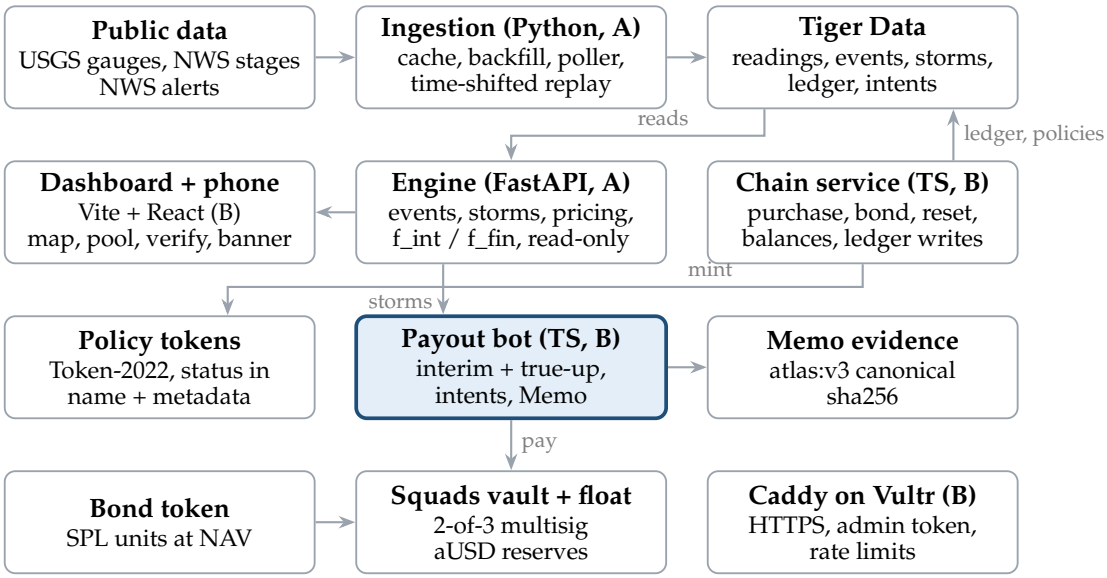
6 to 10 gauges with cached history and a live poller; pricing with confidence bands and a correlated capital charge; atomic purchase; storm-level payouts with interim and true-up; ledger-based pool accounting with per-run reconciliation; bond token with NAV; a time-shifted Harvey replay that runs back to back; a hosted, authenticated dashboard with a phone view.

1.7 Out of scope

Mainnet, real money, a custom Solana program, KYC, a native mobile app, rainfall triggers, property-level flood modelling, cash-out to bank accounts, aUSD token metadata **[V6]**, and any claim to be a licensed insurance product.

2 System architecture

Every money movement is a ledger row and a transaction on chain, and the two reconcile after every run [F2] **[V3]**.



2.1 Data flow for one storm

1. The poller or replay streamer writes readings with `event_time`, `wall_time`, `source` and `run_id` [F9].
2. At Start, the engine records opening balances from `GET /chain/balances` **[V3]**.
3. The engine opens an *event* for a gauge after two consecutive accepted readings at or above `trigger_ft`, if that gauge has no open event in this run [F8]. It latches the value and writes the canonical preimage and hash [F14][F15] **[V2]**.
4. The first event in a run opens a *storm* and sets f_{int}. Later events join the open storm [F7].
5. The bot re-hashes the preimage, then pays every `ACTV` policy on that gauge in force at the

event's `opened_at` **[V14]** the interim amount $f_{\text{int}}B$. Each payment is signed, recorded as an intent, then sent [F12].

6. Each interim payment writes a Memo, sets the policy name and status to `PAID` in the same transaction, and writes a ledger entry.
7. An event closes after 72 hours of event time continuously below trigger. The storm closes when all its events close, or when the replay ends; the engine sets f_{fin}.
8. The bot pays the true-up $(f_{\text{fin}} - f_{\text{int}})B$ to every interim-paid policy, if positive, then marks the storm `trued_up` **[V13]**.
9. Reconciliation: opening balances plus the run's ledger rows equal the balances now **[V3]**.

3 Pre-flight checklist (remaining items)

Jesse (B), now:

- ☐ Devnet SOL for the treasury from faucet.solana.com (GitHub login), or sent by Zeke. Then the setup in `chain/README.md` on devnet; commit `chain/state/devnet.json`.
- ☐ Vultr instance; Caddy; GoDaddy Registry domain pointed at the IP, with the `sslip.io` hostname as fallback [F18].
- ☐ Phone: Phantom in Testnet Mode, funded, screen lock off; phone view URL on the home screen.
- ☐ Read the Vultr, GoDaddy and Devpost multi-track rules [F24].

Zeke (A), now:

- ☐ Tiger Cloud service; TimescaleDB version for the columnstore API [F19]; connection string to Jesse for the server's `.env`.
- ☐ Shared `ADMIN_TOKEN` generated and stored outside the repo [F6].

Already done: monorepo layout, `@solana/web3.js` pinned to 1.x, `docs/api.md`, `docs/evidence.md`, aUSD and Squads scripts, Token-2022 test.

4 Team roles and the API contract

Zeke owns every number and the database. Jesse owns every transaction, every pixel and the servers they run on **[V9]**.

4.1 Roles

Role	Owns
Person A: data and risk (Zeke)	Tiger schema, ingestion, event and storm logic, f_{int} and f_{fin}, replay and run clock, pricing, scenario set, simulated book, ledger reads and reconciliation, FastAPI (read-only), NWS freeze

Person B: chain, product and hosting (Jesse)	Chain service (purchase, bond, reset, balances), all signing, aUSD and policy mints, Squads vault, payout bot, Memo evidence, ledger writes, dashboard and phone view, Vultr, Caddy, domain, deploys, upgrades
Shared	Gates, pitch, Q&A, fallback video, Devpost write-up

Language boundary

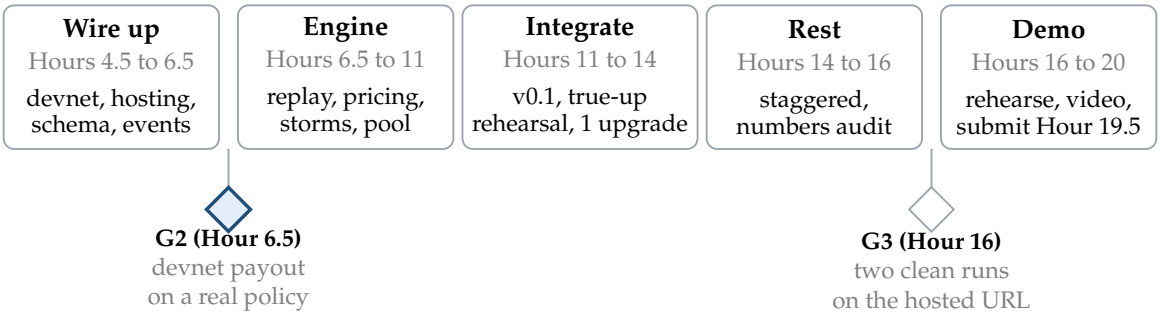
Python never builds or signs a Solana transaction. The chain service and the bot are the only processes with signing keys. FastAPI reads Tiger and serves JSON; the chain service writes policy and ledger rows; the bot writes intents and marks storms `trued_up`.

4.2 API contract (`docs/api.md` is the source of truth) [V1]

Endpoint	Returns	Owner
`GET /replay/status` [V11]	`{run_id, status, clock}`	A
`GET /gauges?run_id=`	`[{gauge_id, name, lat, lon, trigger_ft, latest_ft, status, demo_ready}]`	A
`GET /gauges/{id}/series?run_id=`	`{points: [{event_time, stage_ft}], trigger_ft, rejected_count}`	A
`GET /quote?gauge=&payout=&run_id=`	`{premium, p_payout, band, k, waiting_days, sales_open, reason}`	A
`GET /pool?run_id=`	`{onchain: {seeded_capital, premiums_in, reserve, bond_capital, float, nav_per_unit, payouts_out, reconciled}, simulated: {policies, worst_year_loss, harvey_loss, reserve_used, bond_used, f_int, f_fin, investor_loss_pct}}`	A
`GET /storms?run_id=`	`[{storm_id, status, f_int, f_fin, events: [{event_id, gauge_id, opened_at, closed_at, value, evidence_hash, preimage}], payouts: [{policy, kind, amount, tx_sig}]}]`	A
`GET /stress?scenario=`	`harvey` or `worst_year`: `{aggregate_loss, reserve_used, bond_used, f_fin, investor_loss_pct}`	A
`GET /ledger?run_id=`	Ledger rows and totals by account	A
`POST /replay/start`, `/replay/reset`	`{run_id}`; `X-Admin-Token`; start records opening balances; reset calls `/chain/reset`	A
`GET/POST /api/actions/buy?gauge=&payout=&flow=`	Action metadata; then a Flow A (server-signed) or Flow B (unsigned) transaction	B
`POST /api/purchase/submit`	Flow B: co-signs only a message-hash match [V5]	B
`POST /api/bond/deposit`, `/api/bond/submit`	Wallet signs first; server co-signs, sends, writes the ledger entry	B
`GET /policies?owner=`	`[{policy_mint, gauge_id, gauge_name, payout, status, paid_amount, last_payout}]` [V16]	B
`GET /chain/balances`	`{vault, float, bond_units, ausd_supply, at}` [V3]	B
`POST /chain/reset`	Localhost only, `X-Admin-Token`; refill, statuses, reservations	B

5 Timeline, re-based from Hour 4.5 [V17]

Hour 0 was 1:00 pm CDT, October 10; the deadline is Hour 24. The plan was adopted at Hour 4, so by the late-start rule the lost time comes out of the upgrade block. Jesse's code is done, which frees his hours for devnet, hosting and the phone.



Gates

G1 (now, overdue): every **[VERIFY]** item answered or its fallback in `docs/decisions.md`, including Phantom Flow A. **G2 (Hour 6.5):** Zeke's minimal insert script [F17] opens a storm on devnet; the bot pays a real policy; ledger and chain agree. **G3 (Hour 16, freeze):** on the hosted URL, start, run, reset, run again, twice in a row, no code changes, reconciliation green after each run.

Hours	Zeke (data and risk)	Jesse (chain, product, hosting)
4.5 to 6.5	Tiger schema (6.2); gauges with real site numbers, datum and channel check [F11]; cache Harvey window; event open logic; preimage per `docs/evidence.md` (check the test vector); minimal insert script that opens a storm with f_{int}	Devnet SOL and setup scripts; Vultr, Caddy, domain or `sslip.io`; deploy chain service, bot and web; Phantom Flow A test; precreate policies on demo gauges once Zeke names them
G2 (Hour 6.5): if it fails, cut scope now.		
6.5 to 9	Full replay streamer with time shift; event close; storm close and f_{fin}; `/replay/status`, `/start` (opening balances), `/reset` (calls `/chain/reset`)	Point web at the real engine endpoint by endpoint; reset end to end; reconciliation badge from `/pool`
9 to 11	Pricing, bootstrap band, historical-year scenarios and k; simulated book; stress; NWS freeze with `replay_alerts`; admin token on FastAPI	Bond capital deposit on devnet; backup policy per demo wallet [F21]; rate limits and allowlist checked on the hosted URL
11 to 12	Integration and food. Start, reset, start on the hosted URL. Tag `v0.1`.	
12 to 14	`/docs/numbers.sql`; reconciliation check script	True-up rehearsal with `FORCE_F_INT=0.6` [V8]; one upgrade only: Solana Pay QR (Flow A) or none; verify page on real data
14 to 16	Staggered rest: Zeke rests 14 to 15 while Jesse polishes the phone view; Jesse rests 15 to 16 while Zeke runs the numbers audit.	

Hours	Zeke (data and risk)	Jesse (chain, product, hosting)
G3 (Hour 16, feature freeze)		
16 to 18	Slides: pricing, correlated capital, stress	Two rehearsals each of the 90-second and 4-minute scripts; fallback video
18 to 19.5	README numbers section	Devpost page, screenshots, tracks, address list
19.5 to 24	Submitted. Sleep in shifts; reset and one full run 20 minutes before judging.	

Rule for the whole build

Blocked for 45 minutes: say so out loud and switch to the fallback (section 12). Already cut by the late start: Switchboard, Squads spending limit, Blink unless Flow A passes and time allows.

6 Data layer: Tiger Data

6.1 Sources and checks

- **USGS legacy WaterServices:** instantaneous stage and annual peaks. Being retired (see Sources); cache everything and never call it on stage.
- **USGS new API:** use for the live poller if the legacy service is degraded.
- **NWS flood stages** set `trigger_ft` **[VERIFY]**; **NWS alerts** drive the live sales freeze (7.9) **[VERIFY]**.
- **Cache rule:** every raw response is written to `/data/cache/` before parsing.

Gauge selection: 6 to 10 gauges on major bayous with live data, at least 20 years of peaks and data through Harvey. Read from the stored data which cross `trigger_ft` in the replay window; set `demo_ready = true` on four of them [F22].

Comparability check [F11]: for each gauge, read the datum history and channel projects. Price only from the comparable record and note the decision in `gauges.record_note`.

6.2 Schema

Engine tables (Zeke). The chain tables `policy_pool` and `payout_intents` are in `chain/sql/001_chain_tables.sql`.

```
CREATE TABLE gauge_readings (
  event_time TIMESTAMPTZ NOT NULL,                  -- (possibly shifted) hydrologic time
  wall_time  TIMESTAMPTZ NOT NULL DEFAULT now(),    -- when the row was written
  gauge_id TEXT NOT NULL, stage_ft DOUBLE PRECISION, discharge_cfs DOUBLE PRECISION,
  source TEXT NOT NULL,                             -- 'usgs' | 'replay'
  run_id TEXT NOT NULL DEFAULT 'live',
  rejected BOOLEAN NOT NULL DEFAULT false
);
SELECT create_hypertable('gauge_readings', by_range('event_time'));
```

```
CREATE UNIQUE INDEX ON gauge_readings (gauge_id, event_time, source, run_id);

CREATE TABLE gauges (
  gauge_id TEXT PRIMARY KEY, name TEXT, lat DOUBLE PRECISION, lon DOUBLE PRECISION,
  trigger_ft DOUBLE PRECISION, nws_flood_ft DOUBLE PRECISION,
  record_start INT, record_note TEXT, feed_pubkey TEXT, demo_ready BOOLEAN DEFAULT false
);
CREATE TABLE annual_peaks (gauge_id TEXT, water_year INT, peak_stage_ft DOUBLE PRECISION,
  peak_time TIMESTAMPTZ, PRIMARY KEY (gauge_id, water_year));

CREATE TABLE runs (run_id TEXT PRIMARY KEY, source TEXT NOT NULL,
  started_wall TIMESTAMPTZ NOT NULL, shift INTERVAL, clock TIMESTAMPTZ,
  status TEXT NOT NULL,                              -- running | done | reset
  opening_vault NUMERIC, opening_float NUMERIC);     -- V3: from GET /chain/balances

CREATE TABLE storms (storm_id TEXT PRIMARY KEY, run_id TEXT NOT NULL,
  opened_at TIMESTAMPTZ NOT NULL, closed_at TIMESTAMPTZ,
  f_int NUMERIC, f_fin NUMERIC, status TEXT NOT NULL); -- open | closed | trued_up (bot)

CREATE TABLE gauge_events (
  event_id TEXT PRIMARY KEY, run_id TEXT NOT NULL, storm_id TEXT NOT NULL,
  gauge_id TEXT NOT NULL, opened_at TIMESTAMPTZ NOT NULL, closed_at TIMESTAMPTZ,
  latched_value DOUBLE PRECISION NOT NULL, feed_pubkey TEXT, feed_slot BIGINT,
  preimage TEXT NOT NULL, evidence_hash TEXT NOT NULL);
CREATE UNIQUE INDEX one_open_event ON gauge_events (run_id, gauge_id) WHERE closed_at IS NULL;

CREATE TABLE policies (
  policy_mint TEXT PRIMARY KEY, owner TEXT NOT NULL, gauge_id TEXT REFERENCES gauges,
  payout_usd NUMERIC NOT NULL, premium_usd NUMERIC NOT NULL,
  starts_at TIMESTAMPTZ NOT NULL, ends_at TIMESTAMPTZ NOT NULL,
  status CHAR(4) NOT NULL,                           -- ACTV | PAID | EXPD
  paid_amount NUMERIC NOT NULL DEFAULT 0, demo BOOLEAN NOT NULL DEFAULT false);

CREATE TABLE sim_policies (sim_id BIGINT PRIMARY KEY, gauge_id TEXT REFERENCES gauges,
  payout_usd NUMERIC, premium_usd NUMERIC);

CREATE TABLE ledger (                                -- written by the chain side (V10)
  wall_time TIMESTAMPTZ NOT NULL DEFAULT now(), run_id TEXT NOT NULL,
  entry_id TEXT NOT NULL, account TEXT NOT NULL, amount NUMERIC NOT NULL,
  memo TEXT, tx_sig TEXT);
SELECT create_hypertable('ledger', by_range('wall_time'));

CREATE TABLE replay_alerts (run_id TEXT, gauge_id TEXT, starts_at TIMESTAMPTZ,
  ends_at TIMESTAMPTZ, kind TEXT);

-- Chain side (V12): payout_intents adds last_valid_block_height, attempts,
-- event_id, owner, gauge_id; see chain/sql/001_chain_tables.sql.
```

6.3 Continuous aggregates and columnstore

Unchanged from v3: `gauge_hourly` grouped by `bucket, gauge_id, source, run_id` with `materialized_only = false`; columnstore policy after 7 days with the old compression API as fallback [F19]. History charts read `gauge_hourly`; the live replay chart reads `gauge_readings` for the current run.

6.4 Event logic

One rule for pricing and the contract [F8]: an event opens after two consecutive accepted readings at or above `trigger_ft`; it closes once the gauge has stayed below `trigger_ft` for 72 hours of event time. At most one open event per gauge per run. The open and close queries are unchanged from v3. At the end of a replay the streamer closes all open events and the storm at the run clock. The data-quality guard marks jumps `rejected = true`; the gauge page shows the count.

6.5 Run clock and replay

- **Run clock:** `now()` for a live run; the `event_time` of the last reading written for a replay. Grep for `now()` and `Date.now()` at Hour 10.
- **Window and speed:** 48 hours before the first demo gauge crosses to 24 hours after the last peak; seconds per reading = `REPLAY_SECONDS / count` [F20].
- **Shift:** `event_time = original_time + (started_wall - window_start)`.
- **Policy activity:** a policy is eligible for an event if `starts_at <= opened_at < ends_at` [V14]. Policies run 365 days, so the future-dated replay stays inside the term.

6.6 Reset

`POST /replay/reset` (admin token): stop the streamer and mark the run `reset`; call `POST /chain/reset` with the run id (refill vault and float to the snapshot by minting aUSD, demo policies to `ACTV` on-chain and in Tiger, release reservations, `seeded` ledger rows); then reconcile. Holders keep the aUSD they received; reset mints new test aUSD, so total supply grows. Say so if asked.

7 Pricing and capital model

7.1 Event frequency

$\hat{\lambda}_g^{\text{peak}} = N_g^{\text{peak}}/T_g$ from annual peaks, and $\hat{\lambda}_g^{\text{iv}}$ from instantaneous data with the contract's open/-close rule. Price with the higher of the two [C3]; say which on the slide.

7.2 Premium

$$P(\text{payout}) = 1 - e^{-\hat{\lambda}_g t}, \qquad \mathbb{E}[L] = B\,(1 - e^{-\hat{\lambda}_g t}), \qquad \pi = \mathbb{E}[L](1+\theta) + c\,k.$$

Interim plus true-up never exceed B. Bootstrap 2,000 times for a 5th to 95th percentile band.

7.3 Correlated capital and k

Each historical year is one scenario across all gauges: $S_y = \sum_i B_i\,\mathbf{1}\{\text{peak}_{g(i),y} \geq \text{trigger}_{g(i)}\}$. Required capital $K = \max(\max_y S_y,\ S_{\text{Harvey}}) - \bar{S}$, a worst-observed-year standard, not a modelled tail. Each policy's share is $k_i = KB_i / \sum_j B_j$.

7.4 Waterfall

Each premium is split once: expenses $e\pi$; bond coupon ck, which accrues into NAV (no distributions in the demo); the rest to reserve R. Losses hit R, then bond capital C, then the pay factor.

$$\text{NAV}_t = \frac{C_0 - \text{losses absorbed}_t + \text{coupons accrued}_t}{\text{units outstanding}}.$$

7.5 Storm payouts: interim and true-up

$$f_{\text{int}} = \min\left(1,\ \frac{A}{B\,N_{\text{risk}}}\right), \qquad f_{\text{fin}} = \min\left(1,\ \frac{A}{B\,N_{\text{trig}}}\right) \geq f_{\text{int}},$$

where A is available funds $(R + C)$ at storm open, N_{risk} the active policies on all gauges in the at-risk set, and N_{trig} those whose gauges triggered by storm close. The engine writes f_{int} when the storm opens and f_{fin} when it closes; the bot never computes them. For the true-up rehearsal the engine honours `FORCE_F_INT` **[V8]**.

7.6 Two pool panels

On-chain (devnet): seeded capital labelled "seeded", not premiums [F26]; premiums; reserve; bond capital; float; NAV; reconciliation badge. Simulated book: 2,000 policies; worst-year and Harvey losses; layer use; f_{int} and f_{fin}; investor loss.

7.7 Ledger entries [V4] [V10]

Written by the chain side; each entry sums to zero; the engine reads them and owns the economic split (expenses, coupon, which layer absorbs a loss). Movements outside a running run go to run `session`.

Movement	Rows
Policy purchase	`holders` $-\pi$, `premiums` $+\pi$
Bond deposit	`investors` $-D$, `bond` $+D$
Payout (interim or true-up)	`float` $-x$, `holders` $+x$
Reset refill of the vault	`seeded` $-x$, `reserve` $+x$
Reset refill of the float	`seeded` $-y$, `float` $+y$
Vault to float by proposal	`reserve` $-z$, `float` $+z$

7.8 Reconciliation per run [V3]

At `/replay/start` the engine stores `opening_vault` and `opening_float` from `GET /chain/balances`. After the run, for the vault (`premiums`, `reserve`, `bond` rows) and for the float (`float` rows): opening balance + sum of this run's rows = `GET /chain/balances` now. Absolute totals are never compared, because reset only tops up.

7.9 Underwriting rules

- Waiting period 7 days; demo mode sets 0, and the demo says so.

- **Sales freeze:** live mode reads NWS alerts for the gauge's point; replay mode reads `replay_alerts` for the running run (a flood watch from 24 hours of event time before the first crossing). **When no run is running, sales are open [V7].** The live API is never consulted in demo mode.
- One policy per wallet per gauge; one active reservation per wallet.

8 Solana layer (Jesse)

All signing lives in the chain service and the bot [F16]. Built and tested; the remaining work is devnet, the phone and one optional upgrade.

8.1 aUSD

Classic SPL Token mint, 6 decimals, 1,000,000 to the treasury. No token metadata [V6]: Phantom will show an unnamed token; the phone view is the money moment. Policies use Token-2022 and aUSD the classic Token program; every associated token account is derived with the right program.

8.2 Policy tokens

Supply 1, 0 decimals; NonTransferable, MetadataPointer (to itself), TokenMetadata, and PermanentDelegate (burn at expiry only). Status is a fixed-width code in both the name (`Atlas Policy ACTV`, `Atlas Policy PAID`) and the `status` field, with 256 bytes of rent pre-funded [F13]. Fields: `policy_id`, `gauge_id`, `status`, `terms_hash`, and `uri` to the terms JSON. About 10 pre-created mints per demo gauge.

8.3 Atomic purchase

One transaction: create the buyer's Token-2022 account; `transferChecked` the premium to the vault; `mintTo` one policy; remove the mint authority. The mint is reserved for 2 minutes; one active reservation per wallet.

Flow	How
A (preferred)	The server partially signs as mint authority; the wallet signs as fee payer and sends. Works for the Buy button, Solana Pay and Blinks.
B (fallback)	The server returns the transaction unsigned and stores the hash of its message. The wallet signs; `/api/purchase/submit` co-signs only if the posted message hash matches exactly [V5], then sends. Blinks are dropped.

Phone test (G1): run Flow A from the phone in Phantom Testnet Mode against the hosted service. If Phantom rejects or warns, set `PURCHASE_FLOW=b` and `VITE_PURCHASE_FLOW=b` and record decision #11.

8.4 Pool: Squads multisig

2-of-3 Squads v4; the vault is a PDA, so its aUSD account uses `allowOwnerOffCurve = true`; time lock about 60 seconds in demo mode. The bot float (2,000 aUSD) is pool money held by the bot; it is refilled by reset or by one proposal.

8.5 Payout bot

Per storm: interim $f_{\text{int}}B$ to each `ACTV` policy on a triggered gauge after an on-chain check (holder, status `ACTV`, gauge); true-up $(f_{\text{fin}} - f_{\text{int}})B$ after close; storm marked `trued_up` [V13]. Interim transactions carry the transfer, the Memo and the `PAID` name and status together. If the float is short, the bot warns once and waits.

8.6 Idempotency

1. `intent_id = sha256(policy_mint|storm_id|kind)`; skip if confirmed or in flight.
2. Sign the transaction; the signature is known before sending.
3. Insert the intent with `tx_sig`, `last_valid_block_height` and status `signed` [V12]. Only then send.
4. On every tick, first settle unsettled intents: confirmed → ledger and policy rows exactly once; failed or blockhash expired → `expired`, rebuilt as a new attempt; otherwise wait.

8.7 Evidence [V2]

```
atlas|v3|{gauge_id}|{value}|{event_time}|{run_id}|{storm_id}|{event_id}|{feed}|{slot}
```

- `value`: latched triggering value, exactly 2 decimals. `event_time`: the event's `opened_at`, UTC, whole seconds, trailing `Z`. Python: `strftime("%Y-%m-%dT%H:%M:%SZ")`; TypeScript: `toISOString()` without milliseconds. `feed`, `slot`: Switchboard values or `-`.
- SHA-256 of the UTF-8 preimage, lowercase hex; Memo `atlas:v3:<hash>`. The preimage is stored in `gauge_events`; the bot re-hashes it before paying; the verify page re-hashes it again.
- Test vector: `atlas|v3|08074500|31.46|2026-10-11T03:15:00Z|run-7|run-7-s1|run-7-e3|-|-` hashes to:
  `a3bf1d54bf92482a546d48927da5801cc8452da55dee080956c8ce7222bf3642`

8.8 Bond token

Classic SPL mint. Deposits always use wallet-first signing: the investor signs, the server co-signs on `/api/bond/submit`, sends and writes the ledger entry. NAV comes from the ledger (7.4). Redemption is cut.

8.9 Reset (chain side)

Localhost only, admin token. Mint aUSD into vault and float up to the snapshot (`SNAPSHOT_VAULT`, `SNAPSHOT_FLOAT`); set demo policies to `ACTV` (name and status, three per transaction); mark them

`ACTV` with `paid_amount = 0` in Tiger; release reservations; write `seeded` rows; return balances before and after.

8.10 Upgrades (after G2, only one)

Solana Pay QR on the same buy endpoint, only if Flow A passes (1 hour; drop if not working on the phone). Already cut by the late start: Blink, Squads spending limit, Switchboard.

9 Backend, frontend and hosting

9.1 Services

Service	Role	Owner
FastAPI	Read-only API over Tiger; replay start and reset orchestration; pricing; stress	Zeke
Streamer and poller (Python)	Live polling; replay; event and storm ticks	Zeke
Chain service (TypeScript)	Purchase, bond, Actions, `/chain/reset`, `/chain/balances`, `/policies`	Jesse
Payout bot (TypeScript)	Interim and true-up payouts; intents; Memo	Jesse
Web (Vite + React) [V15]	Dashboard and phone view	Jesse

All run on one Vultr instance under systemd or pm2, behind Caddy. Keep a full local copy runnable on a laptop.

9.2 Frontend

Page	Shows
Map	Gauges coloured normal, near, triggered; demo gauges; storm status; Start and Reset only when the admin token is stored
Gauge	Stage over event time with the trigger line and event markers; rejected count; quote with band, k and freeze status; Buy (Flow A or B)
Pool	On-chain panel with seeded capital labelled and reconciliation badge; simulated book with f_{int} and f_{fin}; payouts by kind
Verify	Payout signature $\rightarrow$ preimage, recomputed hash, Memo hash, match
Phone view	One wallet: balance from chain, "+$500 • PAID • Brays Bayou" banner with the transaction link [V16]
Admin	Stores the admin token in the demo laptop's browser

9.3 Hosting, security and abuse limits

- Caddy terminates HTTPS for the GoDaddy domain; if DNS has not propagated, use the `sslip.io` hostname for the Vultr IP.

- `POST /replay/*` and `/chain/reset` require `X-Admin-Token`; `/chain/reset` accepts only direct loopback connections.
- Buy and bond: 5 requests per minute per IP; demo allowlist (`DEMO_WALLETS`); one active reservation per wallet; reservations expire after 2 minutes.
- Keys only in server environment files: `issuer` and `treasury` for the chain service, `bot` and `issuer` for the bot.

10 Demo script

The moment that wins: the bayou line turns red and the phone view shows the payout at the same instant.

Stage set-up: map on the laptop; phone view mirrored beside it; admin token loaded; fallback video open; one backup policy already bought on a second demo wallet [F21].

10.1 90-second version

0:00 to 0:15	"Maria rents in Meyerland. Federal flood claims take weeks. She needs money in the first 72 hours."
0:15 to 0:35	Tap a demo gauge. Quote: probability, premium, band. Buy on the phone; if it has not confirmed in 10 seconds, switch to the backup wallet.
0:35 to 1:10	Start the replay. The map turns red; the phone view shows "+$500 • PAID".
1:10 to 1:30	Pool page: reserve used, bond absorbs the rest, reconciliation green. "Priced from decades of gauge data in Tiger Data, settled on Solana in seconds."

10.2 4-minute version

0:00 to 0:30	The gap, told through Maria. One sentence on Atlas.
0:30 to 1:15	A demo gauge. Quote with band and history. Buy on the phone (backup ready). "Waiting period is off in demo mode."
1:15 to 2:30	Replay. Map turns red; payouts land; phone shows PAID. Storm closes: "everyone in the storm gets the same share." Pool: reserve used, NAV drops.
2:30 to 3:00	Verify one payout: preimage, hash, Memo match.
3:00 to 3:30	Numbers: worst-year capital need, simulated interim and true-up factors, Tiger compression ratio.
3:30 to 4:00	Limitations in one breath (basis risk, licensing, cash-out), then the close.

Demo rules

Only the four `demo_ready` gauges. Press Reset before every judge group and check the reconciliation badge is green. One full run 20 minutes before judging. Never debug in front of judges: switch to the fallback video within 15 seconds.

11 Judge Q&A preparation

Question	Answer
Your bot decides to pay. What does the chain add?	Purchase is one atomic transaction, so payment cannot be separated from the policy. Every payout carries a Memo hash of the exact reading it acted on. Solvency and every payout are public. A Solana program enforcing the payout rule is the production step.
Is the payout fair in a big storm?	Every triggered policy gets the same interim share at once, sized against all policies that could trigger, and a true-up when the storm closes. Order of crossing does not matter.
Which numbers are real?	On-chain panel: real devnet balances, with starting capital labelled seeded. Simulated panel: a 2,000-policy book, labelled.
How do you know the books are right?	Every money movement is a ledger row. Each run's opening balances plus its ledger rows equal the on-chain balances afterwards; the badge shows it.
How does Maria cash out?	In this build she can't. Off-ramps exist through third parties; a production version would partner with one or pay through a card linked to the wallet [F25].
What if the gauge triggers but my street stayed dry?	Basis risk, the core trade-off of parametric cover. Payouts are small and fixed for immediate needs. Rainfall and multi-gauge triggers are next.
How did you size capital?	Each historical year is a scenario across all gauges, so correlation is built in. Capital covers the worst observed year above the mean: a worst-year standard, not a modelled 1-in-200.
Is your flood history comparable over time?	Datum and channel changes were checked per gauge; pricing uses only the comparable record.
Is this legal?	Selling insurance needs a licence; the bond token is likely a security. Atlas is the engine an insurer, city programme or nonprofit would run.
Can you fake the oracle?	In this build the bot reads Tiger and the replay is served by our server; we say so. An on-chain oracle is the production path.
What stops double payment?	Every payout is signed and recorded before it is sent; a restarted bot settles every recorded signature before building anything new.
Why does aUSD supply keep growing?	Reset mints test aUSD to restore the pool between demo runs. In production there is no reset.

12 Risk register, fallbacks and cut order

Risk	Odds	Switch trigger	Fallback
Devnet faucet refuses	High	No SOL by Hour 5	faucet.solana.com with GitHub login; Zeke sends SOL; last resort: demo on a local validator, said aloud
Engine late for G2	High	No storm row by Hour 6.5	Minimal insert script only; pricing numbers from a notebook for slides
Phantom rejects Flow A	Medium	Phone test	Flow B; drop the QR upgrade
DNS not propagated	Medium	Not resolving by Hour 6	`sslip.io` hostname

Risk	Odds	Switch trigger	Fallback
Reconciliation fails	Medium	Badge red	Fix before G3; check opening balances were recorded at Start
True-up path breaks on devnet	Low	Rehearsal	Already passed locally; demo storm uses $f_{\text{int}} = 1$
Running out of time	High	Missed gate	Cut order below
Live demo failure	Medium	Anything breaks	Fallback video, 15-second rule
Venue network or Vultr fails	Medium	Site unreachable	Local stack, hotspot
USGS legacy degraded	Medium	Failed call	New API; else cache only, live poller dropped and said so
Tiger Cloud limits	Low	Blocking error	Local Docker TimescaleDB

Cut order (first cut first): QR upgrade; GEV fit; verify page; bond deposit flow (seed bond capital by script); true-up beat in the 4-minute script (keep the logic); live poller.

Never cut: time-shifted replay with reset and refill; ledger and reconciliation; admin token; the phone view payout; the map turning red; the pool page; the hosted URL; the backup policy; the fallback video.

13 Limitations of the product

- **Insurance design:** basis risk; fixed benefit for immediate needs only; one payout per term; partial interim payment in a large storm with the rest days later; adverse selection reduced, not removed.
- **Data and oracle:** gauges fail in the largest floods; provisional data can be revised after a payout; single source, itself mid-migration at USGS; record comparability shortens usable history; the replay is served by the team on shifted time; the bot decides to pay, and the Memo makes that auditable, not enforced.
- **Pricing and capital:** few extreme events and wide bands; non-stationarity; capital set to the worst observed year; catastrophe figures from a simulated book; bond holders can lose most of their principal.
- **Legal:** licensed insurer or approved programme required; the bond token is likely a security; disclosure rules apply; payouts depend on a stablecoin.
- **Users:** no cash-out; a lost wallet means a lost policy; people in a crisis may distrust an unfamiliar product.
- **Hackathon build:** devnet and test money; reset mints new aUSD between runs; aUSD has no name in wallets; waiting period off in demo mode; custodial keys; a hot-wallet float; no code audit.

14 Submission checklist

Submit by Hour 19.5; the deadline is 1:00 pm CDT, October 11 (Hour 24).

- ☐ Hosted URL on the GoDaddy domain (or documented `sslip.io`), tested from a phone on mobile data.
- ☐ Admin controls hidden and protected on the public site.
- ☐ Public repository: README with problem, architecture, setup, run, reset and refill, evidence format, limitations.
- ☐ On-chain addresses: aUSD mint, an example policy, multisig and vault, float wallet, bond mint, sample purchase, interim and true-up transactions.
- ☐ Demo video (2 to 3 minutes) with the phone view visible.
- ☐ Slides: Maria, demo, pricing, correlated capital, stress test, Tiger Data usage, Solana usage, limitations, next steps.
- ☐ All seven tracks selected if allowed; each sponsor's usage described separately.
- ☐ `/docs/numbers.sql` reproduces every number on the slides; reconciliation passes.
- ☐ Reset pressed, one full run, badge green, 20 minutes before judging.

15 Unverified assumptions (G1, overdue)

Each needs a yes or its fallback in `docs/decisions.md`. Items already answered on a local validator are marked; recheck them on devnet.

- ☐ Phantom on the phone accepts Flow A [F3]; Phantom shows the policy name.
- ☐ USGS legacy and new API coverage for the chosen gauges; history depth; datum and channel changes [F11].
- ☐ NWS flood stages and alerts by point.
- ☐ Exact Harvey window, reading count and the four gauges that cross trigger [F20][F22].
- ☐ Token-2022 extensions, name and status updates, purchase under 1,232 bytes (passed locally; recheck on devnet).
- ☐ Squads multisig, vault account and time lock (passed locally with the devnet program cloned; recheck on devnet).
- ☐ Tiger Cloud columnstore API [F19]; free-tier limits; whether the track requires Tiger Cloud.
- ☐ Vultr and GoDaddy track rules; Devpost multi-track entry [F24].
- ☐ Domain resolving, or `sslip.io` working with Caddy [F18].

Sources

Carried over from v3 (not re-checked):

- USGS Water Data APIs (legacy WaterServices retirement and possible degradation): https://www.usgs.gov/tools/usgs-water-data-apis

- USGS API decommission notice: https://waterdata.usgs.gov/blog/api-decom-fall-2025/
- Circle testnet faucet (devnet USDC limits): https://developers.circle.com/w3s/docs/developer-console-faucet
- CodeRED: Orion on Devpost (tracks, rules, deadline): https://codered-orion-ccs.devpost.com/